LABORATORIO 8.1: Navegación Multitarea y Arquitectura de Flujo (Login a Dashboard)

1. Contexto y Objetivos de Aprendizaje

Contexto: En el desarrollo Android profesional, una aplicación no es una sola pantalla estática. Los usuarios transitan por múltiples vistas (flujos de autenticación, menús, detalles). Si no gestionamos correctamente la "pila de retroceso" (*Back Stack*), un usuario podría presionar el botón "Atrás" en su dispositivo y volver accidentalmente a la pantalla de Login después de haber ingresado, lo cual es un fallo de seguridad y usabilidad grave.

Objetivos:

1. Implementar la librería de Navigation Compose para estructurar la arquitectura de flujo de la aplicación.
2. Construir un NavHost que gestione múltiples destinos (Auth y Main).
3. Manipular el *Back Stack* utilizando navController.popBackStack() para evitar ciclos de navegación infinitos o retornos no deseados.

2. Requisitos Previos

- Proyecto estandarizado del Laboratorio 1 (SafePass 2026 o proyecto base equivalente) abierto en Android Studio Ladybug.
- Emulador con Android 16 (API 36) configurado y funcional.
- Conocimiento básico de creación de funciones @Composable (visto en semanas anteriores).

3. Descripción General

En este laboratorio, los estudiantes configurarán el componente de navegación de Jetpack Compose. Crearán dos pantallas simuladas: una pantalla de Login y un Dashboard. El núcleo de la práctica consiste en programar la transición entre ambas, destruyendo la pantalla de Login en el proceso para que el flujo sea unidireccional y seguro.

4. Paso a Paso Detallado

Paso 1: Preparación del Entorno (Dependencias)

Antes de usar herramientas de navegación, debemos indicarle a Gradle (nuestro gestor de dependencias) que descargue la librería de Navigation Compose.

1. **Ubicación del archivo:** En el panel izquierdo de Android Studio (vista *Project*), desplegar la carpeta Gradle Scripts y hacer doble clic en build.gradle.kts (Module :app). **(Atención: NO el de nivel de proyecto, sino el del módulo 'app').**
2. **Modificación:** En el bloque dependencies { ... }, agregar la siguiente línea al final:
   Kotlin

```
implementation("androidx.navigation:navigation-compose:2.8.0")
```

3. **Sincronización:** Aparecerá un banner amarillo en la parte superior derecha con el botón **"Sync Now"** (o el ícono de un elefante con una flecha azul). Hacer clic en él y esperar a que la barra de carga inferior termine.

Paso 2: Creación de las Pantallas (Destinos)

Vamos a crear los contenedores visuales de nuestro flujo.

1. **Ruta:** Navega a app/src/main/java/com/tuempresa/tuapp/ (reemplaza por el paquete real de tu proyecto).
2. **Crear paquete:** Haga clic derecho sobre la carpeta principal de su paquete -> New -> Package. Nómbralo ui.screens.
3. **Crear LoginScreen:** Haga clic derecho en la carpeta screens recién creada -> New -> Kotlin Class/File. Nómbralo LoginScreen y seleccionar *File*. Agregar el siguiente código:

```
package com.tuempresa.tuapp.ui.screens

import androidx.compose.foundation.layout.*
import androidx.compose.material3.Button
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp

@Composable
fun LoginScreen(onLoginSuccess: () -> Unit) {
    // POR QUÉ: Recibimos una lambda (onLoginSuccess) como parámetro.
    // Esto desacopla la UI de la lógica de navegación (Mindset Kotlin).
    Column(
        modifier = Modifier.fillMaxSize(),
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Text(text = "Pantalla de Autenticación")
        Spacer(modifier = Modifier.height(16.dp))
        Button(onClick = { onLoginSuccess() }) {
            Text(text = "Ingresar al Dashboard")
        }
    }
}
```

4. **Crear DashboardScreen:** Repite el proceso en la carpeta screens para crear DashboardScreen.kt.

```
package com.tuempresa.tuapp.ui.screens

import androidx.compose.foundation.layout.*
import androidx.compose.material3.Text
```

```
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier

@Composable
fun DashboardScreen() {
    Column(
        modifier = Modifier.fillMaxSize(),
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Text(text = "¡Bienvenido al Dashboard Principal!")
    }
}
```

Paso 3: Arquitectura de Flujo (El NavGraph)

Aquí está el corazón del laboratorio. Centralizaremos la navegación en un solo archivo.

1. **Crear paquete:** Hazer clic derecho en ui -> New -> Package. Nombrarlo navigation.
2. **Crear Archivo:** Clic derecho en navigation -> New -> Kotlin Class/File. Nómbrarlo NavGraph.
3. **Agregar el código:**

```
package com.tuempresa.tuapp.ui.navigation

import androidx.compose.runtime.Composable
import androidx.navigation.compose.NavHost
import androidx.navigation.compose.composable
import androidx.navigation.compose.rememberNavController
import com.tuempresa.tuapp.ui.screens.DashboardScreen
import com.tuempresa.tuapp.ui.screens.LoginScreen

@Composable
fun AppNavigation() {
    // 1. Instanciamos el NavController (El "conductor" de la aplicación)
    val navController = rememberNavController()

    // 2. Definimos el NavHost (El mapa de rutas)
    // startDestination define qué pantalla se carga primero.
    NavHost(navController = navController, startDestination = "auth") {

        // Destino 1: Login
        composable("auth") {
            LoginScreen(
                onLoginSuccess = {
                    // POR QUÉ: Navegamos al 'main', pero limpiamos 'auth' de la pila
```

```
                // para que el usuario no pueda regresar con el botón físico "Atrás".
                navController.navigate("main") {
                    popUpTo("auth") { inclusive = true }
                }
            }
        )
    }

    // Destino 2: Dashboard
    composable("main") {
        DashboardScreen()
    }
  }
}
```

Nota: El popUpTo("auth") { inclusive = true } es el equivalente moderno y declarativo del antiguo comportamiento de popBackStack(). Elimina todo en la memoria hasta llegar a "auth", e incluyéndolo.

Paso 4: Inyección en la Actividad Principal

Todo este flujo debe estar anclado en la puerta de entrada de la aplicación.

1. **Ubicación:** Abre app/src/main/java/com/tuempresa/tuapp/MainActivity.kt.
2. **Reemplazo:** Ubica el bloque setContent { ... } dentro de onCreate. Borrar cualquier función de saludo por defecto (Greeting) y llamar a nuestro AppNavigation(). El resultado debe verse así:

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContent {
        TuAppTheme { // Mantén el nombre del tema de tu proyecto
            // Inyectamos el grafo de navegación
            AppNavigation()
        }
    }
}
```

3. **Ejecución:** Conectar el emulador (API 36) y presionar el botón verde de "Run" (Shift + F10).

---

5. Resultados Esperados (Verificación)

1. Al abrir la app, el emulador debe mostrar "Pantalla de Autenticación" y un botón "Ingresar al Dashboard".
2. Al presionar el botón, la pantalla debe cambiar a "¡Bienvenido al Dashboard Principal!".

3. **Prueba de fuego:** Estando en el Dashboard, si el estudiante presiona el botón "Atrás" del sistema Android (el triángulo o el gesto de deslizar hacia atrás), **la aplicación debe cerrarse en lugar de volver al Login**. Esto confirma que la pila de retroceso fue gestionada correctamente.

---

6. Desafío Extra

Reto para el estudiante: Agrega un botón de "Cerrar Sesión" en el DashboardScreen. Al pulsarlo, el usuario debe regresar a la pantalla de Login ("auth"), asegurándose de destruir la pantalla del Dashboard ("main") de la pila para que no pueda volver a entrar sin credenciales.

1. En DashboardScreen.kt:

```
@Composable
fun DashboardScreen(onLogout: () -> Unit) { // Nueva lambda
  Column(...) {
    Text("¡Bienvenido al Dashboard Principal!")
    Button(onClick = { onLogout() }) { Text("Cerrar Sesión") }
  }
}
```

2. En NavGraph.kt:

```
composable("main") {
  DashboardScreen(
    onLogout = {
      navController.navigate("auth") {
        popUpTo("main") { inclusive = true } // Destruye el dashboard
      }
    }
  )
}
```

---

7. Entregables

- Informe PDF con:
  - **Captura de código:** Archivo NavGraph.kt evidenciando la implementación del popUpTo (o popBackStack()).
  - **Evidencia de:** capturas del emulador donde:
    1. Se presione el botón de Login.
    2. Se ingrese al Dashboard.
    3. Se presione el botón "Atrás" nativo de Android y la aplicación vaya al inicio del sistema (Home) sin pasar de vuelta por el Login.

4. Demás puntos que debe contener el informe como conclusiones, recomendaciones, etc.

8. Criterios de Evaluación

- No olvide revisar la rúbrica de evaluación, en el EVA, para complementar su informe